

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Exp. No: 2

Date: 15-07-2026

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Implementation of Lexical Analyzer using LEX Tool

Aim:

To build a lexical analyzer with the LEX (flex) tool that scans a C++ source program, classifies each token (keywords, identifiers, constants, operators, special characters, comments and function calls), and stores the attributes of every identifier (name, type, number of bytes, value) in a symbol table.

Tools Used:

- Flex
- GCC

Code:

scanner.l

```
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* ----- symbol table ----- */
typedef struct {
    char name[32];
    char type[12];
    int bytes;
    char value[32];
} SymEntry;

SymEntry table[100];
int symCount = 0;
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char activeType[12] = ""; /* type currently being declared on this line */
int lastIdxIndex = -1; /* index of most recently declared identifier */
int awaitingEquals = 0; /* just saw an identifier that could get "= value" */
int awaitingValue = 0; /* just saw "=" after that identifier */

```

```

int sizeOfType(char *t) {
    if (!strcmp(t, "char")) return 1;
    if (!strcmp(t, "bool")) return 1;
    if (!strcmp(t, "float")) return 4;
    if (!strcmp(t, "double")) return 8;
    return 2;
}

```

```

void recordSymbol(char *name) {
    awaitingEquals = 0;
    if (activeType[0] == '\0') { lastIdxIndex = -1; return; }
    strcpy(table[symCount].name, name);
    strcpy(table[symCount].type, activeType);
    table[symCount].bytes = sizeOfType(activeType);
    strcpy(table[symCount].value, "-");
    lastIdxIndex = symCount;
    awaitingEquals = 1;
    symCount++;
}

```

```

void captureValue(char *val) {
    if (awaitingValue && lastIdxIndex >= 0) {

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        strcpy(table[lastIdx].value, val);
    }
    awaitingValue = 0;
    lastIdx = -1;
}

void printSymbolTable(void) {
    printf("\nContent of Symbol Table\n");
    printf("%-15s %-8s %-12s %-10s\n", "Identifier", "Type", "No of bytes", "Value");
    for (int k = 0; k < symCount; k++)
        printf("%-15s %-8s %-12d %-10s\n",
            table[k].name, table[k].type, table[k].bytes, table[k].value);
}

%}

%option nounput noinput
%x COMMENT

DIGIT    [0-9]
LETTER   [a-zA-Z_]
HEX      [0-9a-fA-F]

%%

"/".*    { /* single line comment: ignore */ }

"/*"     { BEGIN(COMMENT); }

<COMMENT>"*/" { BEGIN(INITIAL); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

<COMMENT>\n                { /* stay in comment */ }
<COMMENT>.                  { /* consume comment body */ }
<COMMENT><<EOF>>            {
                                printf("Error: unterminated comment\n");
                                yyterminate();
                                }

```

```

"auto"|"break"|"case"|"catch"|"class"|"const"|"constexpr"|"continue"|"default"|"delete"|"do"|"else"|"
enum"|"explicit"|"extern"|"for"|"friend"|"goto"|"if"|"inline"|"mutable"|"namespace"|"new"|"nullptr"|"o
perator"|"private"|"protected"|"public"|"register"|"return"|"sizeof"|"static"|"struct"|"switch"|"templat
e"|"this"|"throw"|"try"|"typedef"|"typename"|"union"|"unsigned"|"using"|"virtual"|"void"|"volatile"|"w
hile"|"signed" {
                                printf("%s - keyword\n", yytext);
                                activeType[0] = '\0';
                                }

```

```

"int"|"char"|"float"|"double"|"long"|"short"|"bool" {
                                printf("%s - keyword\n", yytext);
                                strcpy(activeType, yytext);
                                }

```

```

^[ \t]*#[^\n]*                { printf("%s - preprocessor directive\n", yytext + strspn(yytext, " \t")); }

```

```

"std::?"cout"                  { printf("%s - output stream (function call)\n", yytext); }

```

```

"std::?"cin"                   { printf("%s - input stream (function call)\n", yytext); }

```

```

"std::?"endl"                  { printf("%s - stream manipulator\n", yytext); }

```

```

"printf"[ \t]*("[^"]*)"         { printf("%s - function call\n", yytext); }

```

```

"scanf"[ \t]*("[^"]*)"          { printf("%s - function call\n", yytext); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

"getch"[\t]*("[^"]*)" { printf("%s - function call\n", yytext); }

"clrscr"[\t]*("[^"]*)" { printf("%s - function call\n", yytext); }

"main"[\t]*("[^"]*)" { printf("%s - function call\n", yytext); }

"true"|"false" { printf("%s - boolean constant\n", yytext); captureValue(yytext); }

{LETTER}{(LETTER){DIGIT}}* {
printf("%s - identifier\n", yytext);
recordSymbol(yytext);
}

"0"[xX]{HEX}+ { printf("%s - hexadecimal constant\n", yytext);
captureValue(yytext); }

{DIGIT}+"."{DIGIT}+ { printf("%s - double constant\n", yytext); captureValue(yytext); }

{DIGIT}+ { printf("%s - integer constant\n", yytext); captureValue(yytext); }

\("[^"\n"]*\)" { printf("%s - string constant\n", yytext); captureValue(yytext); }

\('[^"\\\n\.\.]*\)' { printf("%s - character constant\n", yytext); captureValue(yytext); }

"+="|"-"|"*"|"/"|"%"= " { printf("%s - arithmetic assignment operator\n", yytext);
awaitingEquals = 0; awaitingValue = 0; }

"++"|"--" { printf("%s - unary operator\n", yytext); }

"&&" { printf("&& - logical operator\n"); }

"||" { printf("|| - logical operator\n"); }

"==" { printf("== - relational operator\n"); }

"!=" { printf("!= - relational operator\n"); }

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"<="      { printf("<= - relational operator\n"); }
">="      { printf(">= - relational operator\n"); }
"<<"      { printf("<< - bitwise / stream insertion operator\n"); }
">>"      { printf(">> - bitwise / stream extraction operator\n"); }
"::"      { printf(":: - scope resolution operator\n"); }
"->"      { printf("-> - member access operator\n"); }

[+|-|*|/]  { printf("%c - arithmetic operator\n", yytext[0]); }
[<|>]     { printf("%c - relational operator\n", yytext[0]); }
"="        {
    printf("= - assignment operator\n");
    if (awaitingEquals) { awaitingValue = 1; awaitingEquals = 0; }
}
"!"        { printf("! - logical operator\n"); }
[&|^]      { printf("%c - bitwise operator\n", yytext[0]); }

[;|.|\\|[]|{}|?:] {
    printf("%c - special character\n", yytext[0]);
    if (yytext[0] == ';') activeType[0] = '\0';
}

[ \t\n]+   { /* skip whitespace */ }

.           { /* skip anything unrecognized */ }

%%

int yywrap(void) { return 1; }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
int main(void) {
    yylex();
    printSymbolTable();
    return 0;
}
```

sample.cpp:

```
{
    int a=10,b=20;
    if(a>b)
        cout << "a is greater" << endl;
    else
        std::cout << "b is greater" << std::endl;
}
```

Output:

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

kushaal@Kushaals-MacBook-Air Assignment 2 % ./scanner < sample.cpp
{ - special character
int - keyword
a - identifier
= - assignment operator
10 - integer constant
, - special character
b - identifier
= - assignment operator
20 - integer constant
; - special character
if - keyword
( - special character
a - identifier
> - relational operator
b - identifier
) - special character
cout - output stream (function call)
<< - bitwise / stream insertion operator
"a is greater" - string constant
<< - bitwise / stream insertion operator
endl - stream manipulator
; - special character
else - keyword
std::cout - output stream (function call)
<< - bitwise / stream insertion operator
"b is greater" - string constant
<< - bitwise / stream insertion operator
std::endl - stream manipulator
; - special character
} - special character

```

Content of Symbol Table

Identifier	Type	No of bytes	Value
a	int	2	10
b	int	2	20

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Question Answers

Q1. How does your scanner handle multi-line comments that are not terminated correctly? How does it report errors in such cases?

The scanner handles comments with a Flex exclusive start condition named COMMENT. On reading /*, it enters this state through BEGIN(COMMENT) and silently discards every character until it meets the closing */, where it returns to INITIAL. If the file ends while the scanner is still in the COMMENT state, meaning no */ was found, the <COMMENT><<EOF>> rule fires, prints Error: unterminated comment and stops the scan through yyterminate(). This was tested with a file containing /* this comment never ends and no closing tag: the valid tokens before it were processed correctly and then exactly that error message was printed.

Q2. Construct symbol table for storing the identifier information

The symbol table is an array of SymEntry structures, each holding the name, type, size in bytes (derived from the type: char=1, bool=1, float=4, double=8, otherwise 2) and value. A global "active type" remembers the last type keyword read (for example int), and the next identifier is stored under that type. If an = follows the identifier, the next constant token (integer, hex, double, char or string) is saved as its value. The table is printed once, after the whole file has been scanned, in this format:

Identifier	Type	No of bytes	Value
a	int	2	10
b	int	2	20

9. Learning Outcomes

- Learned to write lexical specifications with regular expressions in Flex.
- Understood Flex's longest-match rule and how the order and specificity of patterns affect tokenization (a bug where if(a>b) was caught by a generic function-call pattern was fixed by restricting that pattern to named functions).
- Used exclusive start conditions (%x) to handle multi-line comments correctly and to detect unterminated ones through <<EOF>>.
- Built a symbol table that records assigned values as well as declarations, using small state flags to connect an identifier with the = and constant that follow it.